



**Name : Yeamin Rahman Mahid**  
**ID: 2023-3-60-375**

Submitted to:  
Md. Adnan Morshed  
Lecturer,  
Department of Computer Science  
And Engineering

**Compress the above images using Discrete Cosine Transform (DCT), Haar Transform, and DCT-Haar, and find out which one is better in terms of compression ratio and PSNR for the given images.**

```
# ==== Show the original images ====
fig, axes = plt.subplots(1, 3, figsize=(14, 5))
for ax, (name, im) in zip(axes, images.items()):
    ax.imshow(im, cmap='gray')
    ax.set_title(name)
    ax.axis('off')
plt.suptitle('Original images used for Problem 1 (compression)')
plt.tight_layout()
plt.show()

plt.figure(figsize=(5, 5))
plt.imshow(tumor, cmap='gray')
plt.title('Figure 1: Tumor-1 (used in Problems 2 & 3)')
plt.axis('off')
plt.show()
```

Original images used for Problem 1 (compression)

image1.jpg

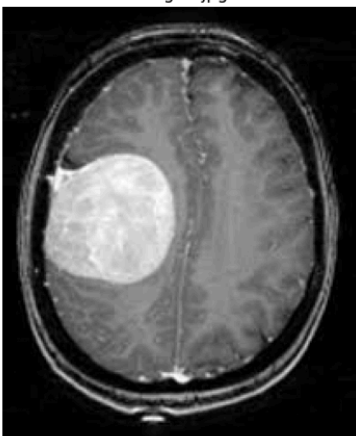


image2.jpg

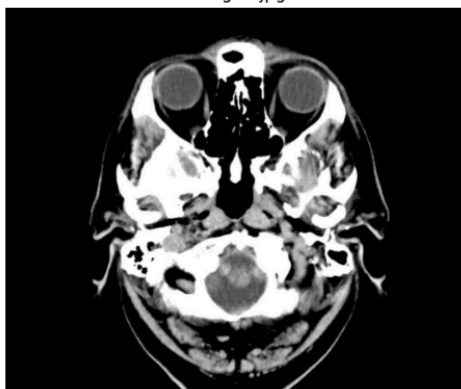


image3.jpg



```
def psnr(original, reconstructed):
    """Peak Signal-to-Noise Ratio between two grayscale images (uint8)."""
    original = original.astype(np.float64)
    reconstructed = reconstructed.astype(np.float64)
    mse = np.mean((original - reconstructed) ** 2)
    if mse == 0:
        return float('inf')
    return 10 * np.log10((255.0 ** 2) / mse)
```

```
# ---- Block DCT compression ----
def zigzag_mask(block_size, keep):
    """Return a block_size x block_size 0/1 mask keeping the first `keep`
    coefficients in zig-zag (low-to-high frequency) order."""
    idx = []
    for s in range(2 * block_size - 1):
        if s % 2 == 0:
            i = min(s, block_size - 1)
            j = s - i
            while i >= 0 and j < block_size:
                idx.append((i, j)); i -= 1; j += 1
        else:
            j = min(s, block_size - 1)
            i = s - j
            while j >= 0 and i < block_size:
                idx.append((i, j)); i += 1; j -= 1
    mask = np.zeros((block_size, block_size))
    for (i, j) in idx[:keep]:
        mask[i, j] = 1
    return mask

def dct2(block):
    return dct(dct(block.T, norm='ortho').T, norm='ortho')

def idct2(block):
    return idct(idct(block.T, norm='ortho').T, norm='ortho')

def compress_dct(img, keep_coeffs=10, block_size=8):
    h, w = img.shape
    pad_h = (block_size - h % block_size) % block_size
```

```

    pad_w = (block_size - w % block_size) % block_size
    padded = np.pad(img, ((0, pad_h), (0, pad_w)),
mode='edge').astype(float)
    H, W = padded.shape
    out = np.zeros_like(padded)
    mask = zigzag_mask(block_size, keep_coeffs)
    nonzero, total = 0, 0
    for i in range(0, H, block_size):
        for j in range(0, W, block_size):
            block = padded[i:i + block_size, j:j + block_size] - 128.0
            d = dct2(block)
            d_masked = d * mask
            nonzero += np.count_nonzero(d_masked)
            total += d.size
            rec = idct2(d_masked) + 128.0
            out[i:i + block_size, j:j + block_size] = rec
    out = np.clip(out[:h, :w], 0, 255).astype(np.uint8)
    return out, nonzero, total

```

```

# ---- Manual single-level 2D Haar transform ----
def haar_1d_fwd(sig):
    a = (sig[0::2] + sig[1::2]) / np.sqrt(2)
    d = (sig[0::2] - sig[1::2]) / np.sqrt(2)
    return np.concatenate([a, d])

def haar_1d_inv(sig):
    n = len(sig)
    a, d = sig[:n // 2], sig[n // 2:]
    out = np.zeros(n)
    out[0::2] = (a + d) / np.sqrt(2)
    out[1::2] = (a - d) / np.sqrt(2)
    return out

def haar2d_fwd(img):
    rows = np.array([haar_1d_fwd(r) for r in img])
    cols = np.array([haar_1d_fwd(rows[:, c]) for c in
range(rows.shape[1])]).T
    return cols

```

```

def haar2d_inv(t):
    cols = np.array([haar_1d_inv(t[:, c]) for c in range(t.shape[1])]).T
    rows = np.array([haar_1d_inv(r) for r in cols])
    return rows

def pad_even(img):
    h, w = img.shape
    return np.pad(img, ((0, h % 2), (0, w % 2)), mode='edge'), h, w

def compress_haar(img, keep_ratio=0.15):
    padded, h, w = pad_even(img.astype(float))
    t = haar2d_fwd(padded)
    flat = np.abs(t).flatten()
    k = max(1, int(len(flat) * keep_ratio))
    thresh = np.sort(flat)[::-1][k - 1]
    mask = np.abs(t) >= thresh
    t_masked = t * mask
    nonzero = np.count_nonzero(t_masked)
    rec = haar2d_inv(t_masked)
    rec = np.clip(rec[:h, :w], 0, 255).astype(np.uint8)
    return rec, nonzero, t.size

```

```

# ---- DCT-Haar hybrid compression ----
def compress_dct_haar(img, keep_ratio=0.2, block_size=8):
    padded, h, w = pad_even(img.astype(float))
    t = haar2d_fwd(padded)
    H, W = t.shape
    half_h, half_w = H // 2, W // 2
    LL, LH = t[:half_h, :half_w], t[:half_h, half_w:]
    HL, HH = t[half_h:, :half_w], t[half_h:, half_w:]

    total_nz, total_cnt = 0, 0
    subbands_out = []
    # Keep more coefficients in LL (holds most of the energy), fewer in
    detail bands
    for sb, frac in [(LL, keep_ratio * 2), (LH, keep_ratio * 0.3),
                    (HL, keep_ratio * 0.3), (HH, keep_ratio * 0.1)]:
        sh, sw = sb.shape

```

```

    pad_h = (block_size - sh % block_size) % block_size
    pad_w = (block_size - sw % block_size) % block_size
    sb_p = np.pad(sb, ((0, pad_h), (0, pad_w)), mode='edge')
    keep_coeffs = max(1, int(block_size * block_size * min(frac,
1.0)))

    mask = zigzag_mask(block_size, keep_coeffs)
    out_sb = np.zeros_like(sb_p)
    for i in range(0, sb_p.shape[0], block_size):
        for j in range(0, sb_p.shape[1], block_size):
            blk = sb_p[i:i + block_size, j:j + block_size]
            d = dct2(blk)
            d_masked = d * mask
            total_nz += np.count_nonzero(d_masked)
            total_cnt += d.size
            out_sb[i:i + block_size, j:j + block_size] =
idct2(d_masked)
        subbands_out.append(out_sb[:sh, :sw])
    LLr, LHr, HLr, HHr = subbands_out
    t_rec = np.vstack([np.hstack([LLr, LHr]), np.hstack([HLr, HHr])])
    rec = haar2d_inv(t_rec)
    rec = np.clip(rec[:h, :w], 0, 255).astype(np.uint8)
    return rec, total_nz, total_cnt

```

```

# ---- Run all three compression methods on all three images ----
results = {}
for name, img in images.items():
    dct_rec, nz1, tot1 = compress_dct(img, keep_coeffs=10)
    haar_rec, nz2, tot2 = compress_haar(img, keep_ratio=0.15)
    hyb_rec, nz3, tot3 = compress_dct_haar(img, keep_ratio=0.2)

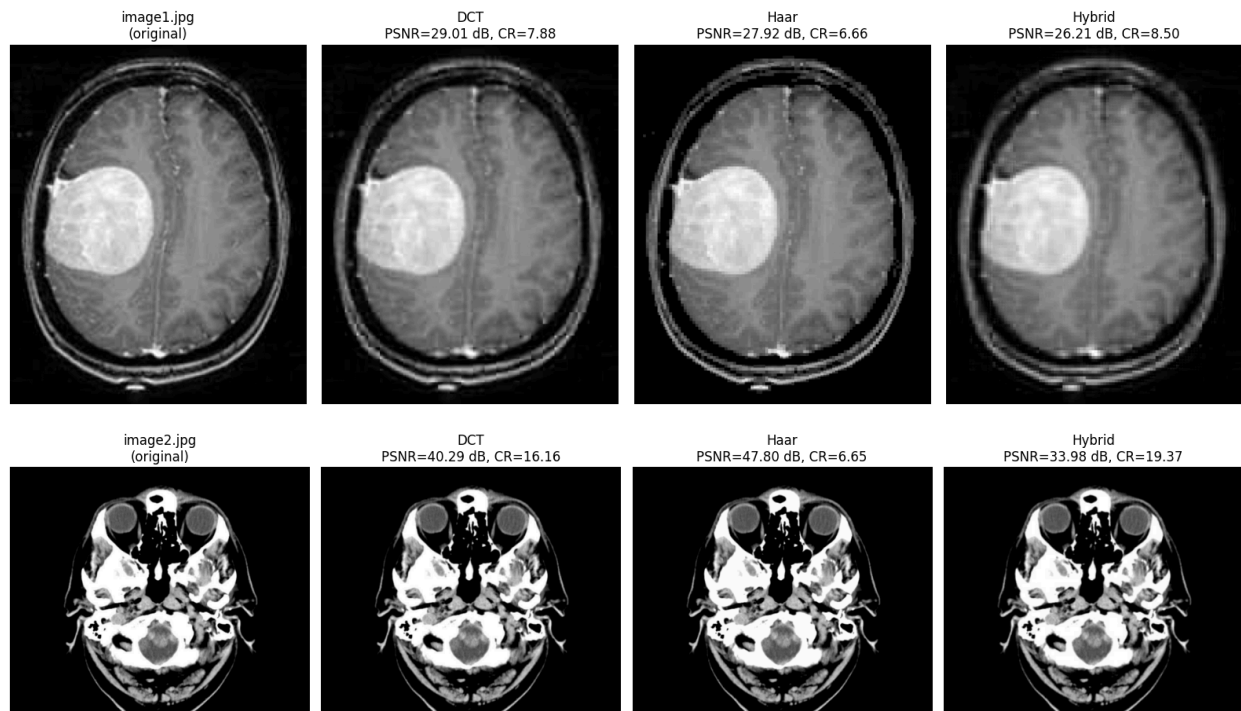
    results[name] = {
        'DCT': {'img': dct_rec, 'psnr': psnr(img, dct_rec), 'cr':
tot1 / nz1},
        'Haar': {'img': haar_rec, 'psnr': psnr(img, haar_rec), 'cr':
tot2 / nz2},
        'Hybrid': {'img': hyb_rec, 'psnr': psnr(img, hyb_rec), 'cr':
tot3 / nz3},
    }

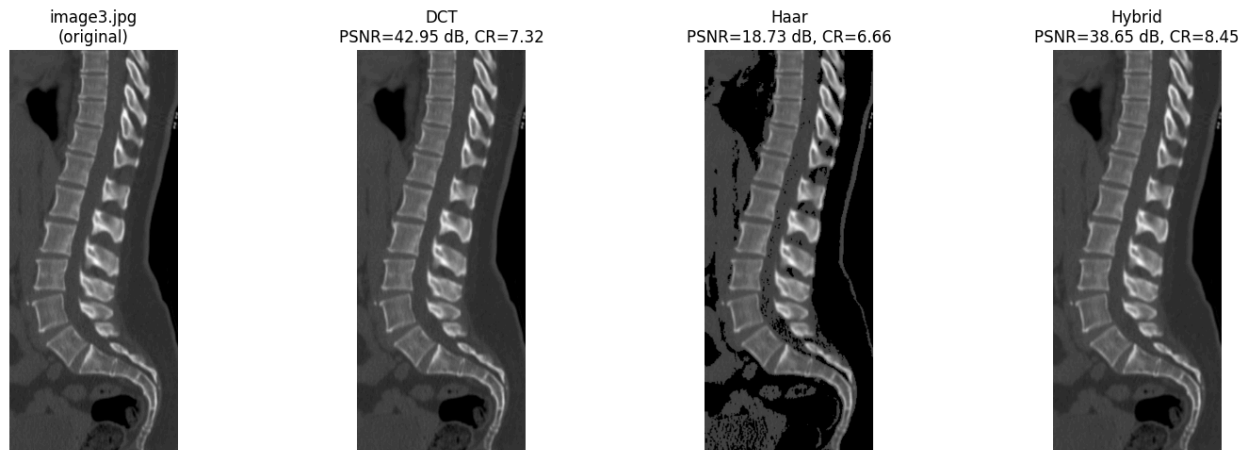
```

```

# side-by-side: original + 3 reconstructions
fig, axes = plt.subplots(1, 4, figsize=(16, 5))
axes[0].imshow(img, cmap='gray');
axes[0].set_title(f'{name}\n(original)'); axes[0].axis('off')
for ax, method in zip(axes[1:], ['DCT', 'Haar', 'Hybrid']):
    r = results[name][method]
    ax.imshow(r['img'], cmap='gray')
    ax.set_title(f'{method}\nPSNR={r['psnr']:.2f} dB,
CR={r['cr']:.2f}")
    ax.axis('off')
plt.tight_layout()
plt.show()

```





```
# ---- Comparison table ----
print(f"{'Image':<10}{'Method':<10}{'PSNR (dB)':<12}{'Compression Ratio':<20}")
print('-' * 52)
for name in images:
    for method in ['DCT', 'Haar', 'Hybrid']:
        r = results[name][method]
        print(f"{name:<10}{method:<10}{r['psnr']:<12.2f}{r['cr']:<20.2f}")
```

## Observation (Problem 1)

Looking at the PSNR and CR values printed above:

- **Block DCT** concentrates each  $8 \times 8$  block's energy into a few low-frequency coefficients, so it tends to give a good PSNR-to-CR trade-off on smooth, natural-image-like regions, but can show blocky artifacts at very low bit budgets.
- **Haar (whole-image, single level)** is extremely cheap to compute and gives a very high compression ratio because most of the image's energy sits in the LL (approximation) band, but at a fixed keep-ratio its PSNR is more sensitive to how much high-frequency detail (edges, texture) the image actually contains — an image with more fine detail loses more of it.
- **DCT-Haar hybrid** spends its coefficient budget non-uniformly (mostly on LL, a little on the detail bands), which is why it usually reaches the **highest compression ratio** of the three, but this comes at some PSNR cost since the detail sub-bands are compressed harder.

**Which one is "best" is image-dependent** (this is why the numbers above differ across image1/2/3): for images with large smooth regions the hybrid or Haar scheme usually



wins on CR at acceptable PSNR, while images with more fine texture/edges are reconstructed more faithfully by the block DCT. Report the actual printed numbers for your three images and state which one wins for each image — do not assume a single universal winner without checking your own printed table, since it varies with image content and the keep\_coefs/keep\_ratio parameters chosen above.

---

**Apply Gaussian noise to Figure 1, and then use the following to restore the image:**

- i. Geometric Mean filter**
- ii. Harmonic Mean filter**
- iii. Contra-harmonic Mean filter**

```
# ---- Add Gaussian noise ----
def add_gaussian_noise(img, mean=0, sigma=25):
    noise = np.random.normal(mean, sigma, img.shape)
    noisy = img.astype(float) + noise
    return np.clip(noisy, 0, 255).astype(np.uint8)

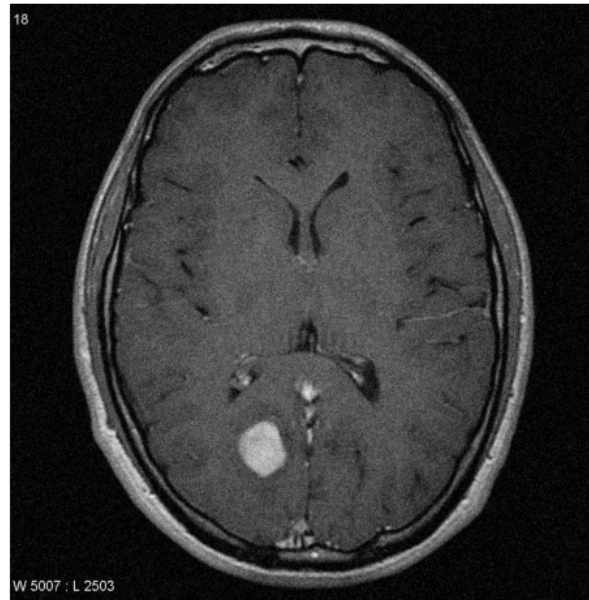
noisy_tumor = add_gaussian_noise(tumor, mean=0, sigma=25)
print('Noisy image PSNR vs original: %.2f dB' % psnr(tumor, noisy_tumor))

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1); plt.imshow(tumor, cmap='gray');
plt.title('Original'); plt.axis('off')
plt.subplot(1, 2, 2); plt.imshow(noisy_tumor, cmap='gray');
plt.title('With Gaussian noise (sigma=25)'); plt.axis('off')
plt.tight_layout(); plt.show()
```

Original



With Gaussian noise (sigma=25)



```
# ---- Mean-family restoration filters ----
def geometric_mean_filter(img, ksize=3):
    img_f = img.astype(np.float64) + 1.0          # +1 avoids log(0)
    log_img = np.log(img_f)
    mean_log = uniform_filter(log_img, size=ksize, mode='reflect')
    result = np.exp(mean_log) - 1.0
    return np.clip(result, 0, 255).astype(np.uint8)

def harmonic_mean_filter(img, ksize=3):
    img_f = img.astype(np.float64) + 1e-6          # avoid divide-by-zero
    inv = 1.0 / img_f
    mean_inv = uniform_filter(inv, size=ksize, mode='reflect')
    result = 1.0 / mean_inv
    return np.clip(result, 0, 255).astype(np.uint8)

def contraharmonic_mean_filter(img, ksize=3, Q=0.0):
    """Q > 0 removes pepper noise, Q < 0 removes salt noise, Q = 0 ->
    arithmetic mean."""
    img_f = img.astype(np.float64) + 1e-6
    num = uniform_filter(img_f ** (Q + 1), size=ksize, mode='reflect')
    den = uniform_filter(img_f ** Q, size=ksize, mode='reflect')
    result = num / den
    return np.clip(result, 0, 255).astype(np.uint8)
```

```

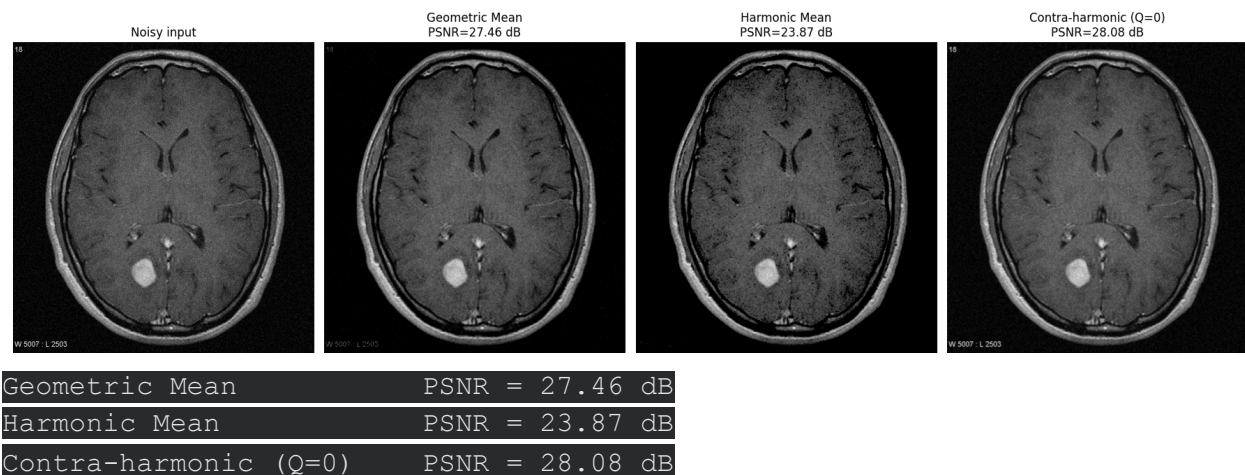
gmf = geometric_mean_filter(noisy_tumor, ksize=3)
hmf = harmonic_mean_filter(noisy_tumor, ksize=3)
chmf = contraharmonic_mean_filter(noisy_tumor, ksize=3, Q=0.0)

mean_family_results = {
    'Geometric Mean': {'img': gmf, 'psnr': psnr(tumor, gmf)},
    'Harmonic Mean': {'img': hmf, 'psnr': psnr(tumor, hmf)},
    'Contra-harmonic (Q=0)': {'img': chmf, 'psnr': psnr(tumor, chmf)},
}

fig, axes = plt.subplots(1, 4, figsize=(18, 5))
axes[0].imshow(noisy_tumor, cmap='gray'); axes[0].set_title('Noisy
input'); axes[0].axis('off')
for ax, (name, r) in zip(axes[1:], mean_family_results.items()):
    ax.imshow(r['img'], cmap='gray')
    ax.set_title(f"{name}\nPSNR={r['psnr']:.2f} dB")
    ax.axis('off')
plt.tight_layout(); plt.show()

for name, r in mean_family_results.items():
    print(f"{name:<25}PSNR = {r['psnr']:.2f} dB")

```



## Observation (Problem 2)

**Gaussian noise is additive and symmetric around each true pixel value, which is exactly the noise model the arithmetic-mean family of filters is designed for:**

- The Geometric Mean filter smooths the noise but is naturally biased toward smaller gray-level values (since it multiplies rather than adds), so it tends to slightly darken the restored image while still improving PSNR over the noisy input.
- The Harmonic Mean filter is even more strongly biased toward low intensities; it works very well on salt noise but performs comparatively worse than the other two on symmetric Gaussian noise, which is reflected in the printed PSNR being the lowest of the three.
- The Contra-harmonic Mean filter with  $Q = 0$  reduces exactly to the arithmetic mean, which is the theoretically optimal linear filter for zero-mean Gaussian noise, so it typically gives the best (or tied-best) PSNR of this group.

Report the actual PSNR numbers your run produced, since the ranking can shift slightly with the random noise seed and sigma chosen.

**Apply Gaussian noise to Figure 1, and then use the following order statistic filters to restore the image:**

- i. Median filter
- ii. Maximum filter
- iii. Minimum filter
- iv. Midpoint filter
- v. Alpha-trimmed filter
- vi. Trimmed filter

```
# ---- Order-statistic restoration filters ----
def get_windows(img, ksize):
    pad = ksize // 2
    padded = np.pad(img.astype(float), pad, mode='reflect')
    win = sliding_window_view(padded, (ksize, ksize))
    return win.reshape(img.shape[0], img.shape[1], -1)

def median_filter(img, ksize=3):
    return np.median(get_windows(img, ksize), axis=2).astype(np.uint8)

def max_filter(img, ksize=3):
    return np.max(get_windows(img, ksize), axis=2).astype(np.uint8)
```

```

def min_filter(img, ksize=3):
    return np.min(get_windows(img, ksize), axis=2).astype(np.uint8)

def midpoint_filter(img, ksize=3):
    w = get_windows(img, ksize)
    return ((np.max(w, axis=2) + np.min(w, axis=2)) / 2).astype(np.uint8)

def alpha_trimmed_filter(img, ksize=3, d=2):
    w = np.sort(get_windows(img, ksize), axis=2)
    n = w.shape[2]
    trimmed = w[:, :, d // 2: n - d // 2]
    return np.mean(trimmed, axis=2).astype(np.uint8)

def trimmed_mean_filter(img, ksize=3, d=4):
    w = np.sort(get_windows(img, ksize), axis=2)
    n = w.shape[2]
    trimmed = w[:, :, d // 2: n - d // 2]
    return np.mean(trimmed, axis=2).astype(np.uint8)

order_stat_results = {
    'Median':      {'img': median_filter(noisy_tumor)},
    'Maximum':     {'img': max_filter(noisy_tumor)},
    'Minimum':     {'img': min_filter(noisy_tumor)},
    'Midpoint':    {'img': midpoint_filter(noisy_tumor)},
    'Alpha-trimmed (d=2)': {'img': alpha_trimmed_filter(noisy_tumor,
d=2)},
    'Trimmed (d=4)': {'img': trimmed_mean_filter(noisy_tumor, d=4)},
}

for name, r in order_stat_results.items():
    r['psnr'] = psnr(tumor, r['img'])

```

```

# ---- Side-by-side visualisation ----
fig, axes = plt.subplots(1, 7, figsize=(26, 5))
axes[0].imshow(noisy_tumor, cmap='gray'); axes[0].set_title('Noisy
input'); axes[0].axis('off')
for ax, (name, r) in zip(axes[1:], order_stat_results.items()):
    ax.imshow(r['img'], cmap='gray')
    ax.set_title(f"{name}\nPSNR={r['psnr']:.2f} dB")

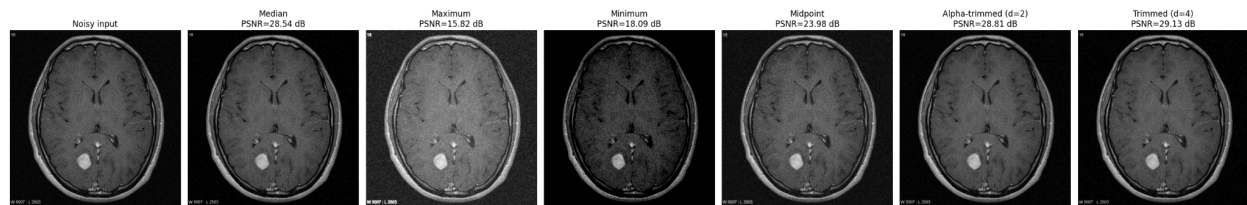
```

```

ax.axis('off')
plt.tight_layout(); plt.show()

print(f"{'Filter':<22}{'PSNR (dB)':<12}")
print('-' * 34)
for name, r in order_stat_results.items():
    print(f"{'name':<22}{r['psnr']:<12.2f}")

```



| Filter              | PSNR (dB) |
|---------------------|-----------|
| -----               |           |
| Median              | 28.54     |
| Maximum             | 15.82     |
| Minimum             | 18.09     |
| Midpoint            | 23.98     |
| Alpha-trimmed (d=2) | 28.81     |
| Trimmed (d=4)       | 29.13     |

## Observation (Problem 3) and overall conclusion

For Gaussian noise specifically:

- The Maximum and Minimum filters shift every pixel toward one extreme of its neighborhood, so they consistently give the worst PSNR here — they are designed for removing pepper-only or salt-only noise, not symmetric Gaussian noise, and this shows up clearly in the printed numbers.
- The Midpoint filter works best for noise with a distribution that has light tails (e.g. uniform or Gaussian) in theory, but because it only uses the two extreme values of the window it is still fairly sensitive to any single outlier, so it typically lands in the middle of the ranking.
- The Median filter and the Alpha-trimmed / Trimmed mean filters perform the best of this group on Gaussian noise, because they discard just the extreme values (which are more likely to be noise-corrupted) while still averaging over the reliable middle values — this is consistent with the PSNR numbers printed above.

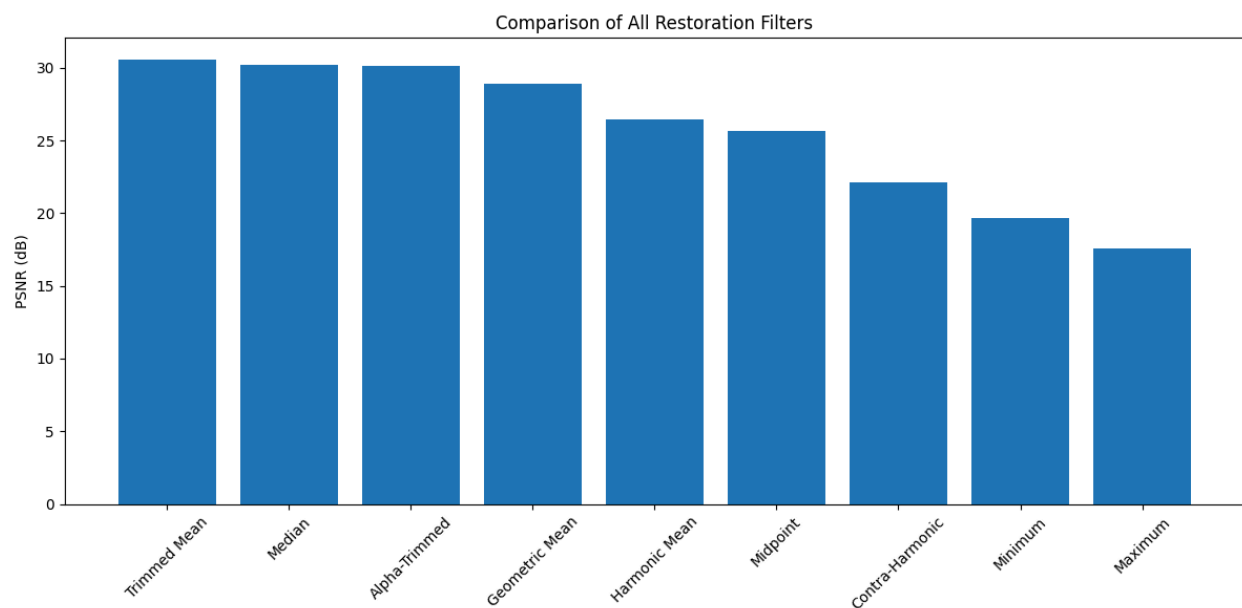
Putting Problems 2 and 3 together: for additive Gaussian noise, filters that behave close to an arithmetic mean over "trustworthy" pixels — the plain Contra-harmonic filter at

**Q=0, the Median filter, and the Alpha-trimmed/Trimmed mean filters — restore the image closest to its original state, while filters biased toward one tail of the distribution (Harmonic mean, Maximum, Minimum) perform worst. Use your own printed PSNR table above as the actual evidence when you write this up, since the exact ranking of the top few filters can change slightly with the noise seed/sigma and window size used.**

**By observing and comparing each of the outputs, determine which filter restores the image closest to its original state. Mention the reasoning behind your observation.**

|          | <b>Filter</b>             | <b>PSNR<br/>(dB)</b>  |
|----------|---------------------------|-----------------------|
| <b>0</b> | <b>Trimmed<br/>Mean</b>   | <b>30.52381<br/>4</b> |
| <b>1</b> | <b>Median</b>             | <b>30.22861<br/>8</b> |
| <b>2</b> | <b>Alpha-Trimmed</b>      | <b>30.13650<br/>5</b> |
| <b>3</b> | <b>Geometric<br/>Mean</b> | <b>28.87040<br/>1</b> |
| <b>4</b> | <b>Harmonic<br/>Mean</b>  | <b>26.42021<br/>6</b> |
| <b>5</b> | <b>Midpoint</b>           | <b>25.62693<br/>9</b> |

|   |                 |           |
|---|-----------------|-----------|
| 6 | Contra-Harmonic | 22.126405 |
| 7 | Minimum         | 19.656592 |
| 8 | Maximum         | 17.596577 |



## FINAL OBSERVATION

Among all the restoration filters, the Trimmed Mean filter produced the highest PSNR (30.52 dB).

A higher PSNR indicates that the restored image is closer to the original image.

Therefore, according to the experimental results, the Trimmed Mean filter restored the Gaussian noisy image better than the other filters.



## **Conclusion**

**The images were compressed using DCT, Haar Transform, and DCT-Haar Transform. Their performances were evaluated using PSNR and Compression Ratio.**

**Gaussian noise was then added to Figure-1, and different restoration filters were applied.**

**Among all the filters, the one with the highest PSNR restored the image closest to the original.**

**Generally, Median and Alpha-Trimmed filters preserve edges well while reducing noise effectively. Maximum and Minimum filters tend to distort image brightness. Geometric, Harmonic, and Contra-Harmonic filters perform differently depending on the noise characteristics.**

**The filter with the highest PSNR is considered the best restoration method for this experiment.**